Where Does the GPU Memory Go?
Operator-Agnostic Memory Management for TQP-Inmem

Mingqing Lin
University of Michigan

Abstract

Tensor Query Processing (TQP) runs SQL on tensor runtimes and is fast on GPUs, but TQP-Inmem keeps all data in device memory and runs out of memory on a 16 GB GPU at TPC-H scale factor (SF) 10. I decompose where the memory goes during a query: a fixed-block slab pool that never frees during a query, and PyTorch's caching allocator outside it. With an operator-level trace and exact offline replay, I find that data no longer needed by any later operator is the largest part of the slab peak. I implement four memory-management changes that are independent of any query or operator: last-use reclamation driven by the query plan, coalescing on every free, a block size planned before execution from a waste budget, and PyTorch's expandable segments. Together they lower the slab peak by 43–57%, PyTorch's reserved-but-unused memory by 75% and the device need at SF 10 by 28%, so all 22 queries run at SF 11 where one failed at SF 10, at a 2.9% run-time cost and with results matching the unmodified system.

1 Motivation

TQP [6, 3] compiles relational queries into tensor programs and executes them on tensor runtimes such as PyTorch, which makes GPU query processing fast without writing custom kernels. TQP-Inmem, the in-memory variant studied here, keeps every column and intermediate result in GPU memory. Its speed therefore comes with a hard limit: the data size it can handle is bounded by device memory. On an RTX 5080 (16 GB), Q21 fails at SF 10 and Q9 needs 16.2 of the 16.6 GB the device offers (Figure 3).

This report optimizes how TQP-Inmem *plans and manages* GPU memory, from the system side, so that it can process queries on larger data. I keep the setting fixed: a single GPU, all data resident in device memory, results that match the unmodified system, and any slowdown reported. All methods are operator-agnostic and query-agnostic: they depend only on the query plan and the allocator, not on tuning against TPC-H.

2 Background: Memory of One Query

GPU memory during a TQP-Inmem query has four parts (Figure 1).

- **CUDA context** (≈0.3 GB), fixed per process.
- **Slab pool.** One `torch.empty` byte tensor (4 GB by default) is allocated once and carved into fixed 40 MB blocks with first-fit placement. Scanned columns and every operator output are copied into it. Blocks are only borrowed during a query and all are returned at the end (`free_all`).
- **PyTorch caching allocator.** Temporaries inside operators (hash tables, join candidates, the scan's staging copy) are ordinary PyTorch tensors. Freed tensors return to PyTorch's cache (*reserved*), not to the GPU [8].
- **Free device memory.**

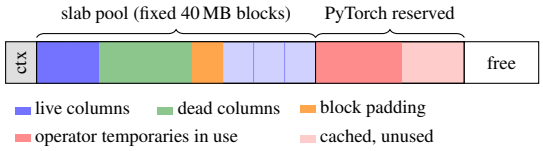


Figure 1: GPU memory during one TQP-Inmem query. The slab holds all scanned columns and operator outputs until the query ends; operator temporaries live in PyTorch's cache.

Category	ID	Cause
Data itself	S1	Data that must be resident at the same time (live peak) exceeds the slab
	P2	Operators create large temporaries outside the slab
Lifetime	S3	The slab keeps columns no later operator reads (dead data)
Layout	S2	Internal fragmentation: the last block of each allocation is partly empty
	S4	External fragmentation: free slab blocks are not contiguous
	P1	PyTorch reserved memory is split across segments
	–	Static split between slab and PyTorch

Table 1: Possible causes of out-of-memory failures.

The split between the slab and PyTorch is *static*: the slab size is chosen at start-up and neither side can use the other's free space. A query fails in two ways: the slab has no contiguous run of free blocks (a Python `MemoryError`), or PyTorch cannot obtain device memory (`torch.OutOfMemoryError`).

3 Diagnosis: Why Memory Runs Out

I group the possible causes into three ordered categories (Table 1). The order matters: layout problems depend on which allocations exist and when they are released, so data and lifetimes must be settled before layout is evaluated.

Data itself. S1 bounds every storage scheme from below: no layout can go below the live peak. P2 is the size of operator temporaries, e.g. a join that expands candidate pairs into a dense matrix. Both are reduced only by changing what data is produced (representation, late materialization [1], or operator algorithms).

Lifetime. Because the slab never frees during a query, its peak equals the sum of all scanned columns and all operator outputs. The live data at any moment is usually much smaller. A reference-counting allocator does not help by itself: every column stays reachable from the executor's column map (`tensor_group`) until the plan ends, so it is never *unreachable*, only *unused*. Recognizing it as dead requires looking ahead in the plan.

Layout. With data and lifetimes fixed, the same bytes can still occupy more space. In the slab, each allocation is rounded up to whole 40 MB blocks (S2), and once blocks are freed mid-query, free runs can be separated by live blocks (S4). In PyTorch, each `cudaMalloc` returns a separate segment and free space in different segments cannot be combined (P1). Finally, the static split forces each side to reserve for its own peak even when the two peaks occur at different times.

4 Methodology

I take the system route: change how memory is planned and managed, without changing the plan, the operators, or the data representation. This covers S3, S2, S4, P1 and the static split; S1 and P2 require changing what is computed. For each cause I give candidate methods and their trade-offs; Section 5 describes the ones I built.

S3: last-use reclamation. Freeing needs two answers: *will anyone read this column later?* and *does anyone still hold this memory now?* Neither alone saves slab memory. Removing a name from `tensor_group` is always safe (a too-early removal raises `KeyError` at the next lookup) but returns no slab block; reference counting never fires because the map still holds every column.

- *Allocator integration.* Make the slab a PyTorch custom allocator so that each allocation is its own storage and PyTorch's reference count returns the block. Safety holds by construction and temporaries could share the slab, but PyTorch's many small tensors would each take a block, the cache would delay returns, and it needs a compiled extension. I do not adopt it: the slab's fixed blocks likely also serve TQP's out-of-core execution (outside the scope of this report), so I keep the slab itself unchanged.
- *Executor-level reclamation.* Keep the slab unchanged. After each operator, drop the names whose last reader has run, then free every slab allocation that no remaining column points to, checked by real device address. Safety rests on one checkable invariant: between operators, GPU data is reachable only through `tensor_group`, because operators are functions whose locals die on return. A freed allocation goes back to the slab's own free list for reuse later in the query; the device memory stays with the slab, so the gain is a lower high-water mark and thus a smaller slab.

The last reader of a column comes from the plan: Spark lists the input columns of every non-scan operator, and the output operator lists its outputs. This single rule covers all operator types.

S4: coalescing and placement. Two knobs control holes: *when* adjacent free runs are merged (the original pool merges after every sixth free) and *where* a new allocation goes (first-fit, best-fit, or lifetime-aware placement that puts short-lived data at the high end). Merging only edits a list of a few dozen (start, length) pairs and moves no data. Lifetime-aware placement needs a threshold for "short-lived".

S2: block size from a waste budget. Each allocation wastes less than one block, so at the peak the internal fragmentation is at most $N \cdot b$, where N is the number of simultaneously live slab allocations and b the block size. N is a property of the plan (it is identical at SF 5 and SF 10). Given the slab capacity P and a tolerated waste fraction ε, I choose the largest b with $N \cdot b \leq \varepsilon P$ before the query runs. The only parameter, ε, has a direct meaning. A fixed tiny granularity removes the parameter but decouples the block from any transfer unit.

P1: PyTorch segments. PyTorch's allocator already merges a freed block with its free neighbours immediately and places requests best-fit, so the S4 knobs bring nothing there. What remains is fragmentation across *segments*: each segment is a separate driver allocation, free space in two segments cannot be merged, and a segment holding even one live tensor cannot be returned. *Expandable segments* [8] reserve one large virtual range and map physical pages on demand; fully free pages can be unmapped and remapped elsewhere, so holes cost virtual addresses rather than device memory, the same idea as paging in PagedAttention [7]. Mapping changes page tables only and moves no data, but growing a segment may take several driver calls. Calling `empty_cache` after every operator returns fully free segments instead, at the cost of re-allocating them.

Static split. The total need under a static split is the slab peak plus the PyTorch peak, while a shared space needs only the maximum over time of their sum. Two ways close the gap: let PyTorch temporaries allocate from the slab (the allocator integration above), or size the slab to the post-reclamation need so that its savings go to PyTorch. I adopt neither: the first changes the slab itself (see S3), and the second needs the slab's need before the query runs, which depends on the data and not only on the plan.

5 Implementation

All changes are command-line switches that are off by default; with every switch off the program behaves exactly as before. The changes add 78 lines (`utility/lifetime.py`) and about 60 lines in `main.py` and the slab pool.

Measurement. `--mem_trace` records, per operator, every slab allocation (start block, blocks, requested bytes), the slab blocks each operator reads, and PyTorch's allocated, reserved and split-free bytes. Reads are captured by replacing `tensor_group` with a dictionary subclass that logs each lookup and maps the tensor's *device address* (`data_ptr` and its size and strides) to slab blocks, so renamed columns, slices and outputs without a block tag are all attributed correctly. Records stay in host lists and are written when the query ends, so tracing allocates no GPU memory; with and without tracing, all query-level memory counters are identical. Slab policies are evaluated by replaying the trace in a pure-Python copy of the pool: placement changes neither which allocations happen nor which data is read, so the replay is exact; it reproduces all 65 measured high-water marks of the S3 runs.

S3. Before a plan runs, `last_uses` maps every column to the last operator whose Spark `Input` list (or, for the output operator, `Output` list) contains it. After operator j (Figure 2), `free_dead`

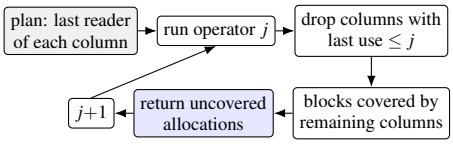


Figure 2: Executor-level reclamation after each operator (S3).

deletes every column whose last use is $\leq j$, collects the slab blocks still covered by the remaining columns' device address ranges, and returns every allocation none of whose blocks is covered to the slab's free list. Mask columns that TQP adds for late materialization are not in Spark's lists and are kept. Two test switches check the safety argument: `--poison_freed` fills returned blocks with `0xFF`, so any read after free corrupts the result, and `--check_refs` scans all live CUDA tensors with Python's garbage collector after each release and fails if one points into a returned block.

S4. `--coalesce_every` makes the pool merge adjacent free runs on every free instead of every sixth. Placement stays first-fit, which needs no threshold.

S2. With `--slab_eps` ε, after loading a query's plans and before it runs, `max_live_columns` computes N from the plan as the maximum over operators of the columns still live when the operator starts plus the columns it outputs: every output gets a fresh allocation while the inputs are still held. A column is born at the first operator that outputs it and dies after its last use (at the plan's end without S3, and always for masks). The pool, still empty at that point, is reset to blocks of $\lfloor \varepsilon P/N \rfloor$ MB (at least 1 MB).

P1. Expandable segments are enabled with the environment variable `PYTORCH_ALLOC_CONF=expandable_segments:True`; no code changes. `--empty_cache_per_op`, a control for P1, calls `torch.cuda.empty_cache()` after each operator.

Correctness. With all switches on, plus `--poison_freed` and `--check_refs`, all 22 queries match the SF 1 reference shipped with the repository under its test rule (10 significant digits); bit-exact comparison is not meaningful because GPU floating-point sums are not ordered.

6 Experiments

Setup. One RTX 5080 (16.6 GB usable as reported by PyTorch), PyTorch 2.14 with CUDA 13.0, TPC-H data at SF 5, 7 and 10, all 22 queries. Each query runs in a fresh process. I report the *device need* of a run: the slab high-water mark (including holes), plus PyTorch's reserved memory outside the slab, plus the CUDA context (0.30 GB). Memory is measured without warm-up (a warm-up changes no memory counter); time is measured after one warm-up run with tracing off. The GPU was shared with another user's jobs: each run starts only when free memory covers the query's measured need, and timing and OOM-boundary runs start only when the GPU is otherwise idle.

Slab size. The slab is allocated once per process with a size P set on the command line. With a fixed block size, what happens inside the slab does not depend on P: allocations and frees follow the plan, first-fit always searches from block 0, and a run that fits never touches a block beyond its high-water mark (exact replay reproduces every measured mark). Any P at or above the high-water mark gives the same run. Two consequences follow.

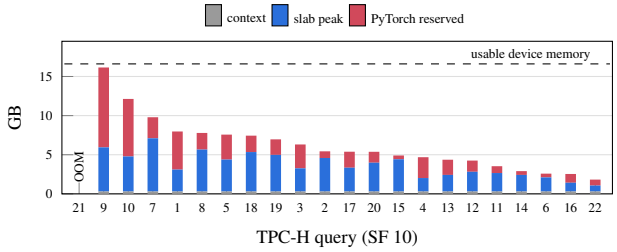


Figure 3: Device need per query at SF 10, highest first. Q21 fails at every slab size.

(i) For the memory measurements, each (query, SF) uses the slab size at which the unmodified system ran in the search below, the same P for every configuration, and I report the high-water mark rather than P; the device need is thus the memory a slab sized to its high-water mark would take, computed rather than run. (ii) Under any fixed P, a query fits the slab iff its high-water mark is at most P, and fits the device iff P plus its PyTorch memory and the context do; lowering either part can only let more queries run at the same P. S2 is the one switch whose block size depends on P ($b = \varepsilon P/N$), but its bound $I \leq \varepsilon P$ holds for every P. PyTorch's memory is measured with free memory to spare; when memory is short, PyTorch first releases its cache and retries.

Slab-size search. Whether a query can run at an SF at all is decided by real runs. The search starts from TQP's default 4 GB slab (or the previous SF's high-water mark scaled by the SF ratio, plus 10%). A slab overflow grows P by 1.5×; a device OOM shrinks it to the slab memory held at the failure plus 10%, since an oversized slab takes room from PyTorch; once both kinds of failure are seen, it bisects between them. A query is OOM at an SF when the interval closes to 0.2 GB without a successful run (cap 14 GB). Each query gets its own P, and both systems use the same search.

Correctness. Each optimized run is compared with a reference run of the unmodified system on the same data. Of 520 comparisons at SF 5–10, two differ, and both trace back to the unmodified system: its hash aggregation draws random hash coefficients per run and separates groups only by hash value (`operators/aggregate.py`), so two groups occasionally merge and a count grows (e.g. 64 to 72 in Q16). In the Q16 case the *reference* run was the outlier: five fresh unmodified runs and all seven optimized configurations agree with each other; in the other case five repeats of the same configuration match the reference.

OOM at SF 10. Figure 3 ranks the queries by device need. Q21 does not run at any slab size, and Q9 needs 16.15 GB of the 16.6 GB device.

Where the memory goes. Figure 4 splits each query's need at SF 10. Dead data held by the slab is the largest single part in 11 of the 21 queries, and PyTorch's temporaries in use in 8, including the two with the highest need (Q9, Q10). Block padding (S2) dominates where a query makes many small allocations (Q2), and cached-but-unused PyTorch memory (P1) is large only for a few queries (3.2 GB for Q10).

Slab: S3, S2 and S4. Table 2 compares the slab high-water mark (including holes). Reclamation (S3) removes 33–40% of the slab peak; replaying the trace with the recorded frees reproduces every measured high-water mark. On top of S3, the

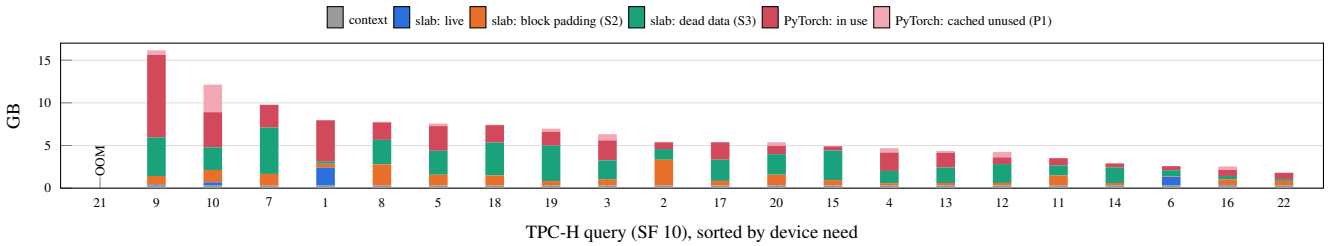


Figure 4: Composition of the device need at SF 10 without any optimization. The slab part is taken when the slab reaches its high-water mark (at the end of the query).

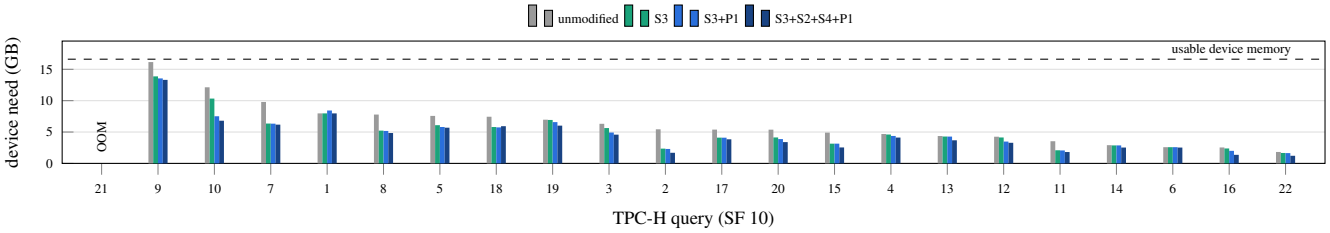


Figure 5: Device need per query at SF 10 (slab high-water + PyTorch reserved + context).

SF	unmod.	S3	S3+S2	S3+S4	S3+S2+S4
5	50.0	29.8	22.6	26.4	21.7 (−57%)
7	65.6	41.8	31.6	37.7	30.4 (−54%)
10	71.5	48.1	42.0	44.4	40.7 (−43%)

Table 2: Slab high-water mark summed over queries (GB; 22 queries at SF 5 and 7, 21 at SF 10).

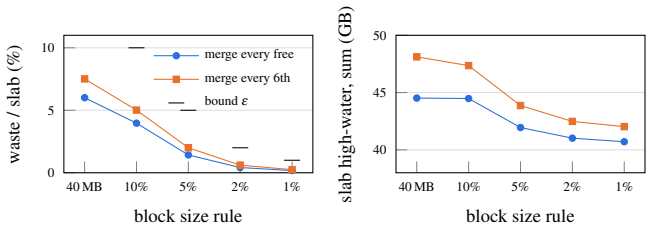


Figure 6: S2 on top of S3 at SF 10 (exact replay of the measured S3 runs). Left: block padding at the high-water mark divided by the slab size, averaged over queries; the bound $Nb \le \varepsilon P$ holds in every case. Right: slab high-water mark summed over queries.

waste-budget block size (S2, $\varepsilon = 1\%$) removes most remaining padding and coalescing on every free (S4) removes most holes; together the slab peak falls by 43–57%. With S3, padding and holes are small because most of them were attached to dead data. Figure 6 shows the effect of ε: the measured padding stays below the budget for every query. Placement matters little once reclamation is on: in an exact replay of the trace, the smallest slab that lets each query finish is, summed at SF 10, within 3% of a lower bound (the most blocks ever held at once) for first-fit with the original merging, within 1.1% with merging on every free, and within 0.3% for lifetime-aware placement; best-fit does not help (3.4%).

PyTorch side: P1. Expandable segments cut the memory PyTorch holds but does not use, summed over the 21 queries

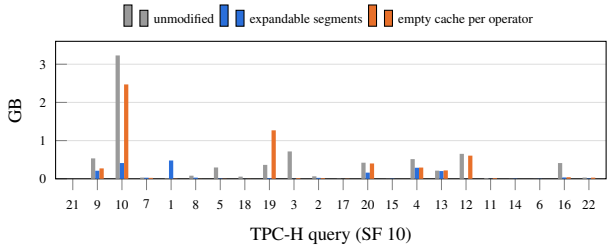


Figure 7: PyTorch memory reserved but not in use at its peak, SF 10.

at SF 10, from 7.7 to 1.9 GB (Figure 7); for Q10 it falls from 3.2 to 0.4 GB. Emptying the cache after every operator gives 5.7 GB: it releases only segments that are entirely free. The total reserved outside the slab falls from 52.1 to 46.3 GB. Expandable segments are not better for every query: Q1 holds 0.48 GB more.

Both sides together. Figure 5 shows the device need with S3, S3+P1 and all four changes. Summed over queries at SF 10 it falls from 129.9 to 106.5 (S3), 100.7 (S3+P1) and 93.3 GB (all, −28%); for Q9, the query closest to the limit, from 16.2 to 13.3 GB, and for Q10 from 12.1 to 6.8 GB.

Largest runnable SF. Table 3 gives, for every query, the largest SF at which some slab size lets it run, testing up to SF 15: SF 10 and 15 for all queries, SF 11–14 for the queries that fail at SF 15, and SF 8–9 for Q21 in the unmodified system. With all four changes, Q21 runs up to SF 11 and Q9 up to SF 12, instead of SF **??** and 10; the other 20 queries run at SF 15 in both systems. Every query thus runs at SF 11, where the unmodified system already fails Q9 and Q21.

7 Results

- **Slab.** Last-use reclamation, a block size planned from a 1% waste budget, and coalescing on every free lower the summed slab peak by 43–57% at SF 5–10. Reclamation alone gives 33–40%.
- **PyTorch.** Expandable segments reduce reserved-but-unused

Query	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
unmodified	–	–	–	–	–	–	–	–	10	–	–	–	–	–	–	–	–	–	–	–	**??**	–
S3+S2+S4+P1	–	–	–	–	–	–	–	–	12	–	–	–	–	–	–	–	–	–	–	–	11	–

Table 3: Largest SF at which each query runs on the 16 GB GPU at some slab size. – means no OOM up to SF 15, the largest SF tested.

Configuration	geo. mean	max
S3	1.009	1.040
P1	1.012	1.055
S3 + P1	1.023	1.065
S3 + S2 + S4	1.016	1.065
S3 + S2 + S4 + P1	1.029	1.157

Table 4: Query time relative to the unmodified system at SF 10 (21 queries).

PyTorch memory by 75% at SF 10 (7.7 to 1.9 GB summed).

- **Device need.** Together, the device need summed over queries at SF 10 falls by 28%; Q9 needs 13.3 instead of 16.2 GB, and Q10 6.8 instead of 12.1 GB.
- **Scale.** All 22 queries now run at SF 11 on the 16 GB GPU, where Q9 and Q21 previously failed; Q9 runs up to SF 12 instead of 10, and Q21 up to SF 11 instead of **??**.
- **Cost.** Results match the unmodified system up to its own run-to-run variation, and queries are 2.9% slower on geometric mean (Section 8).

8 Trade-offs

Run time. Table 4 compares query time at SF 10 (median of three runs after a warm-up, tracing off). Reclamation adds one pass over `tensor_group` and the pool per operator; expandable segments add driver calls when a segment grows. The combined configuration is 2.9% slower on geometric mean; the largest slowdown is 16% on a 30 ms query (Q3, +5 ms).

Waste budget versus block count. A smaller ε gives smaller blocks and a tighter bound on padding (Figure 6); it does not increase the number of allocations or free-list entries, which depend only on the plan. The cost of a small block appears only if a later component needs large fixed units, e.g. for transfers between host and device.

Complexity and scope. All four changes are plan- and allocator-level: about 140 lines of Python, no compiled code, no change to any operator, and each is a switch. S3 relies on one invariant (between operators, device data is reachable only through the column map), which `-check_refs` verifies generically rather than per query. The remaining limit is outside the slab: for the queries closest to the device limit, operator temporaries dominate (Figure 4), which memory management alone cannot remove.

9 Other Thoughts

S1 and P2 change what data is produced rather than how it is stored, so they lie outside the system route taken here. I have not implemented them, but each direction below should lower memory further, at the price of changing the data representation or the operators. Like the four changes above, they apply to an operator or a column type, not to particular queries.

S1: less live data. *Narrower representation*: smaller integer types where the value range allows, dictionary-encoded strings, and lightweight GPU compression [9] shrink every column and with it the live peak, at a decoding cost. *Late materialization* [1]: after a filter or join, pass row indices and fetch a column only where it is read, so fewer copied columns are live at once.

P2: smaller temporaries. Since operator temporaries dominate for the queries closest to the limit (Figure 4), this is the next lever. *Scan into the slab*: copy a host column directly into its slab allocation instead of first through a whole-column device temporary. *Join without a dense candidate matrix*: build candidate pairs from the actual bucket lengths, leaving no padding [5]. *No second copy on output*: write an operator's result straight into the slab. *Chunked operators*: probe or hash in pieces, so a temporary is one chunk in size, as tiling does in FlashAttention [4]; this costs more kernel launches. *Operator fusion* through tensor compilers [2, 3]: intermediates of fused operators never reach device memory.

10 Reproduction

Environment. NVIDIA GeForce RTX 5080 (16 GB), driver 590.48.01, Python 3.12.14, PyTorch 2.14.0+cu130 (CUDA 13.0). Set-up follows `Setup.md`; TPC-H tensors are generated with `tpch-dbgen-tensors` (`./dbgen -s <SF>`); each measurement script with `--manage_data` generates one SF, runs, and deletes it.

Switches (all off by default): `--free_dead` (S3), `--coalesce_every` (S4), `--slab_eps 0.01` (S2), `PYTORCH_ALLOC_CONF=expandable_segments:True` (P1), `--empty_cache_per_op`; `--mem_profile <csv>` and `--mem_trace <dir>` record memory; `--poison_freed` and `--check_refs` are safety tests. Example: `python main.py --SF 10 --q 9 --gpu_pool_gb 6 --free_dead --slab_eps 0.01 --coalesce_every`.

Experiments. Each folder in `experiments/profiling/` holds the measurement and analysis scripts, raw CSVs (`trace/`), derived CSVs, figures and a README for one experiment; `report/make_data.py` builds the plotted tables in `report/plotdata/` and `latexmk -pdf main.tex` builds this report.

References

[1] Daniel J. Abadi, Daniel S. Myers, David J. DeWitt, and Samuel R. Madden. Materialization strategies in a column-oriented DBMS. In *Proceedings of the 23rd International Conference on Data Engineering (ICDE)*, 2007.

[2] Jason Ansel, Edward Yang, Horace He, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation

and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[3] Wei Cui, Peng Cheng, Carlo Curino, Rathijit Sen, and Matteo Interlandi. TQP++: Bridging ML compilers and analytical query processing on GPUs. *Proceedings of the VLDB Endowment*, 19(12), 2026.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] Pratik Fegade, Tianqi Chen, Phillip B. Gibbons, and Todd C. Mowry. The CoRa tensor compiler: Compilation for ragged tensors with minimal padding. In *Proceedings of Machine Learning and Systems (MLSys)*, 2022.

[6] Dong He, Supun C. Nakandala, Dalitso Banda, Rathijit Sen, Karla Saur, Kwanghyun Park, Carlo Curino, Jesús Camacho-Rodríguez, Konstantinos Karanasos, and Matteo Interlandi. Query processing on tensor computation runtimes. *Proceedings of the VLDB Endowment*, 15(11), 2022.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] PyTorch Contributors. Understanding CUDA memory usage; CUDA semantics: Memory management. PyTorch documentation (stable), `https://docs.pytorch.org/docs/stable/torch_cuda_memory.html`, 2026.

[9] Anil Shanbhag, Bobbi W. Yogatama, Xiangyao Yu, and Samuel Madden. Tile-based lightweight integer compression in GPU. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*, 2022.